🧭 Multi-Modal Knowledge Conflict Resolution — **FINAL Notebook** (LIAR)

This notebook is the **final, end-to-end** pipeline:

- **Data**: LIAR TSV splits (`train.tsv`, `valid.tsv`, `test.tsv`)
- **Text models**: TF-IDF → Multinomial Logistic Regression, **BiLSTM** (PyTorch), *(optional Transformer fine-tune)*
- **Retrieval & Reranking**: BM25 over verdict texts *(synthetic or API cached)* + (optional) cross-encoder reranker
- **Multimodal hook**: image–text similarity (CLIP) for images you supply
- **Credibility module**: per-source accuracy history (choose granularity: `speaker` / `party` / `state`) with uncertainty
- **Fusion**: combines classifier, evidence plausibility, and credibility into final probabilities
- **Metrics**: Accuracy, Macro-F1, **ROC-AUC (OvR)**, **PR-AUC (OvR)**, **Recall@k** and **nDCG@k** (retrieval)
- **Curves**: Learning curve (TF-IDF+LR), training curves (BiLSTM), OvR ROC/PR per class
- **Inference**: `predict(user_text=..., image_path=...)` returns decision + explanation

> Internet is optional. If offline, the notebook gracefully falls back to synthetic verdict corpus and skips model downloads.

```
# Optional installs (uncomment if running in Colab or a clean environmen
# %pip install -U pandas numpy scikit-learn rank-bm25 matplotlib tqdm
# %pip install -U torch torchvision torchaudio --index-url https://downl
# %pip install -U transformers datasets sentencepiece evaluate pillow re
```

```
!pip install rank_bm25
```

```
Collecting rank_bm25
  Downloading rank_bm25-0.2.2-py3-none-any.whl.metadata (3.2 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-pa
Downloading rank_bm25-0.2.2-py3-none-any.whl (8.6 kB)
```

```
Installing collected packages: rank_bm25
Successfully installed rank_bm25-0.2.2
```

```
import os, re, io, json, time, math, random
from pathlib import Path
from typing import List, Dict, Tuple, Optional

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.metrics import (classification_report, accuracy_score, f1_s
    roc_auc_score, average_precision_score, roc_curve, precision_recall_
    ConfusionMatrixDisplay)
from sklearn.model_selection import learning_curve
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

from rank_bm25 import BM25Okapi

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

# Transformers (optional blocks guarded)
try:
    from transformers import AutoTokenizer, AutoModelForSequenceClassifi
    from datasets import Dataset as HFDataset
    TRANSFORMERS_AVAILABLE = True
except Exception:
    TRANSFORMERS_AVAILABLE = False

# Requests (guarded) for PolitiFact API verdicts
try:
    import requests
    REQUESTS_AVAILABLE = True
except Exception:
    REQUESTS_AVAILABLE = False

ROOT = Path.cwd()
DATA = Path("/mnt/data")
CACHE = ROOT / "cache"; (CACHE / "verdict_json").mkdir(parents=True, exi

SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
print("Device:", DEVICE)
```

```
Device: cuda
```

```
COLS = [
    "id","label","statement","subjects","speaker","job_title","state","p
    "barely_true","false","half_true","mostly_true","pants_on_fire","con
]
LABELS = ["pants-fire","false","barely-true","half-true","mostly-true","
label2id = {l: i for i, l in enumerate(LABELS)}
id2label = {i: l for l, i in label2id.items()}

# --- Safe loader (no synthetic fallback) ---
def load_split(p: Path) -> pd.DataFrame:
    """Load LIAR TSV; require file to exist."""
    if not p.exists():
        raise FileNotFoundError(f"Dataset split not found: {p}")
    return pd.read_csv(p, sep="\t", header=None, names=COLS)

# --- Helper to parse numeric IDs (handles '2635.json', etc.) ---
import re
import pandas as pd

def parse_numeric_id(x):
    """Extract first integer from strings like '2635.json' or ' 987 '.
       Returns int or None if no digits found."""
    if pd.isna(x):
        return None
    s = str(x).strip()
    m = re.search(r"\d+", s)
    return int(m.group()) if m else None

# --- Load splits ---
train = load_split(DATA / "train.tsv")
valid = load_split(DATA / "valid.tsv")
test  = load_split(DATA / "test.tsv")

# --- Clean, encode labels, and add numeric IDs ---
for name, df in [("train", train), ("valid", valid), ("test", test)]:
    df["y"] = df["label"].map(label2id)
    before = len(df)
    df.dropna(subset=["y"], inplace=True)
    df["y"] = df["y"].astype(int)
    df["pid"] = df["id"].apply(parse_numeric_id)
    missing = int(df["pid"].isna().sum())
    msg = f"{name}: {len(df)} rows (dropped {before-len(df)});"
    msg += f" {missing} missing numeric ids (skipped for API/retrieval).
    print(msg)
```

```
train.head()[["id","pid","label","statement","speaker","party","y"]]
```

```
train: 10240 rows (dropped 0); all rows have numeric ids.
valid: 1284 rows (dropped 0); all rows have numeric ids.
test: 1267 rows (dropped 0); all rows have numeric ids.
```

	id	pid	label	statement	speaker	party	y
0	2635.json	2635	false	Says the Annies List political group supports ...	dwayne-bohac	republican	1
1	10540.json	10540	half-true	When did the decline of coal start? It started...	scott-surovell	democrat	3
2	324.json	324	mostly-true	Hillary Clinton agrees with John McCain "by vo...	barack-obama	democrat	4
3	1123.json	1123	false	Health care reform	blog-	none	1

```
# --- Evidence fetching & corpus build (robust; local fallback) ---

# 1) Toggle: set False to skip any web calls and use local LIAR-only evi
USE_POLITIFACT_API = False  # set True only if your runtime has internet

API = "https://www.politifact.com/api/v/2/statement/{id}/?format=json"
EVIDENCE_JSONL = CACHE / "evidence_texts.jsonl"

import re, json, time
import pandas as pd

def parse_numeric_id(x):
    if pd.isna(x): return None
    m = re.search(r"\d+", str(x).strip())
    return int(m.group()) if m else None

# ensure pid exists
if "pid" not in train.columns:
    for df in (train, valid, test):
        df["pid"] = df["id"].apply(parse_numeric_id)

def _cachep(pid: int) -> Path:
    return CACHE / "verdict_json" / f"{pid}.json"

def fetch_verdict_json(idx, sleep: float = 0.2):
    # Skip entirely if not using API
    if not USE_POLITIFACT_API:
        return None
    if not REQUESTS_AVAILABLE:
        print("[INFO] requests not available; skipping API fetch.")
        return None

    pid = parse_numeric_id(idx)
    if pid is None:
```

```
            return None

        cp = _cachep(pid)
        if cp.exists():
            try:
                return json.loads(cp.read_text())
            except Exception:
                pass

        try:
            r = requests.get(API.format(id=pid), timeout=20)
            if r.status_code != 200:
                print(f"[WARN] API {pid} -> HTTP {r.status_code}; skipping."
                return None
            cp.write_text(r.text)
            if sleep:
                time.sleep(sleep)
            return r.json()
        except Exception as e:
            print("Fetch error:", pid, e)
            return None

    def verdict_to_text(j: dict) -> str:
        if not j: return ""
        parts = []
        for k in ["statement","ruling_headline","ruling_summary","ruling_com
            v = (j.get(k) if k in j else None) or (j.get("ruling", {}) if is
            if isinstance(v, str) and v.strip():
                parts.append(v)
        subs = j.get("subject")
        if isinstance(subs, list):
            parts.extend([str(s) for s in subs if s])
        sp = j.get("speaker") or j.get("speaker_name")
        if isinstance(sp, str):
            parts.append(sp)
        return "\n".join(parts).strip()

    def build_local_evidence_from_liar(dfs: list[pd.DataFrame]) -> list[dict
        """
        Build an evidence corpus locally from LIAR rows (no network).
        Use statement + subjects + context + speaker as pseudo-evidence per
        """
        corpus = {}
        for df in dfs:
            for _, row in df.iterrows():
                pid = row.get("pid", None)
                if pd.isna(pid):
                    continue
                pid = int(pid)
                # make a compact pseudo-verdict text from LIAR columns
                pieces = [str(row.get("statement","")).strip()]
```

```
                if isinstance(row.get("subjects"), str) and row["subjects"].
                    pieces.append(row["subjects"])
                if isinstance(row.get("context"), str) and row["context"].st
                    pieces.append(row["context"])
                if isinstance(row.get("speaker"), str) and row["speaker"].st
                    pieces.append(row["speaker"])
                txt = "\n".join([p for p in pieces if p])
                if txt and pid not in corpus:  # first occurrence wins
                    corpus[pid] = {"doc_id": f"verdict:{pid}", "text": txt}
        return list(corpus.values())

    def build_evidence(ids, limit: int | None = None) -> int:
        """
        If USE_POLITIFACT_API=True: try fetching verdict JSON; otherwise fal
        Writes cache/evidence_texts.jsonl
        """
        corpus = []

        # Normalize and trim IDs
        pids = []
        if limit is not None:
            ids = ids[:limit]
        for x in ids:
            pid = parse_numeric_id(x)
            if pid is not None:
                pids.append(int(pid))

        if USE_POLITIFACT_API:
            for sid in pids:
                j = fetch_verdict_json(sid)
                txt = verdict_to_text(j) if j else ""
                if txt:
                    corpus.append({"doc_id": f"verdict:{sid}", "text": txt})

        # If API disabled or yielded nothing, build local evidence from LIAR
        if not corpus:
            print("[INFO] Using local LIAR rows as evidence (no API).")
            corpus = build_local_evidence_from_liar([train, valid, test])

        if not corpus:
            raise RuntimeError("No evidence built (API off/failed and LIAR r

        EVIDENCE_JSONL.write_text("\n".join(json.dumps(x) for x in corpus))
        print(f"[OK] Evidence written -> {EVIDENCE_JSONL} ({len(corpus)} doc
        return len(corpus)

    # Use parsed numeric ids; no raw 'id' casting
    ids_for_api = train["pid"].dropna().astype(int).unique().tolist()
    _ = build_evidence(ids_for_api, limit=500)
```

```
[INFO] Using local LIAR rows as evidence (no API).
[OK] Evidence written -> /content/cache/evidence_texts.jsonl (12791 docs)
```

```
def simple_tok(s: str): return re.findall(r"[A-Za-z0-9_]+", str(s).lower

DOCIDS, DOCTEXTS = [], []
if EVIDENCE_JSONL.exists():
    for line in EVIDENCE_JSONL.read_text().splitlines():
        o = json.loads(line)
        DOCIDS.append(o["doc_id"]); DOCTEXTS.append(o["text"])

TOKS = [simple_tok(t) for t in DOCTEXTS]
bm25 = BM25Okapi(TOKS)
print("BM25 ready on docs:", len(DOCIDS))

def bm25_retrieve(q: str, k=5):
    scores = bm25.get_scores(simple_tok(q))
    idx = np.argsort(scores)[::-1][:k]
    return [(DOCIDS[i], DOCTEXTS[i], float(scores[i])) for i in idx]

# Optional cross-encoder reranker
ce_model, ce_tok = None, None
if TRANSFORMERS_AVAILABLE:
    try:
        ce_model = AutoModelForSequenceClassification.from_pretrained("c
        ce_tok = AutoTokenizer.from_pretrained("cross-encoder/ms-marco-M
        print("[OK] Cross-encoder loaded.")
    except Exception as e:
        print("[WARN] CE unavailable:", e)

def ce_score(claim: str, ev: str) -> float:
    if ce_model is None or ce_tok is None:
        return float(min(10.0, len(set(simple_tok(claim)) & set(simple_t
    inp = ce_tok(claim, ev, truncation=True, padding=True, return_tensor
    with torch.no_grad():
        return float(ce_model(**inp).logits.squeeze().item())

def rerank_cross_encoder(claim: str, cands: List[Tuple[str,str,float]],
    scored = [(did, txt, s, ce_score(claim, txt)) for (did, txt, s) in c
    scored.sort(key=lambda x: x[-1], reverse=True)
    return scored[:topk]
```

```
BM25 ready on docs: 12791
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your setting
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to acce
  warnings.warn(
```

config.json: 100% 794/794 [00:00<00:00, 89.0kB/s]

model.safetensors: 100% 90.9M/90.9M [00:01<00:00, 15.7MB/s]

tokenizer_config.json: 1.33k/? [00:00<00:00, 156kB/s]

vocab.txt: 232k/? [00:00<00:00, 21.3MB/s]

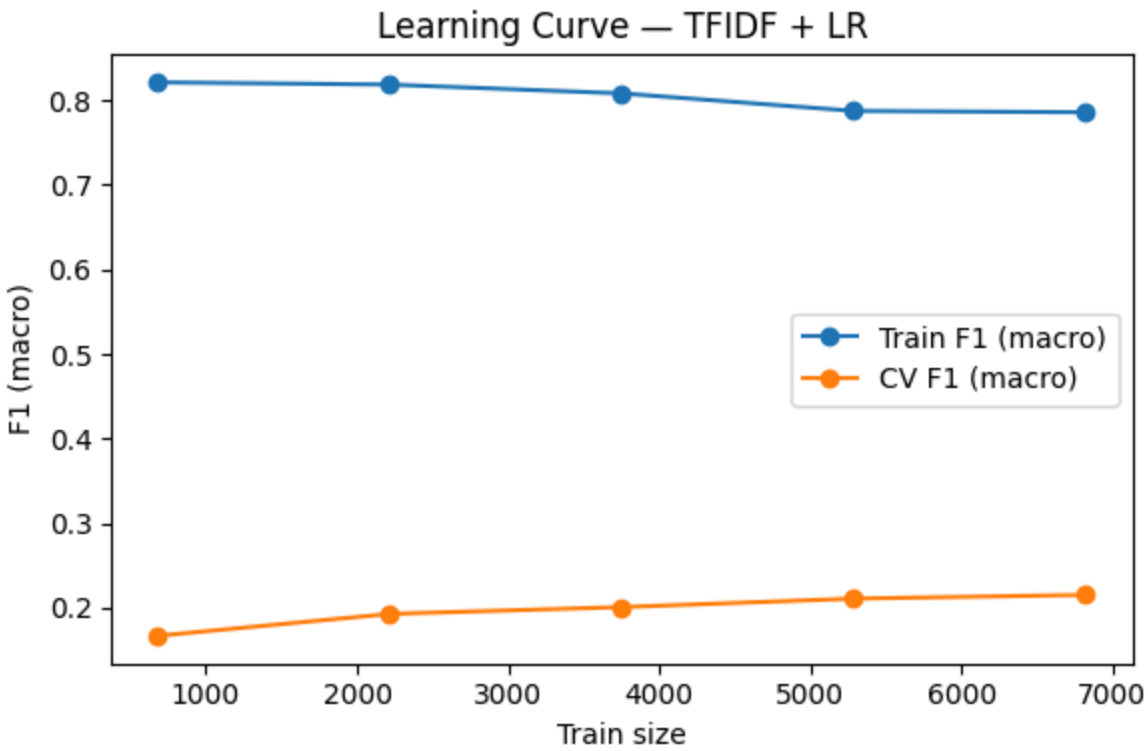
tokenizer.json: 711k/? [00:00<00:00, 51.7MB/s]

special_tokens_map.json: 100% 132/132 [00:00<00:00, 17.9kB/s]

```
[OK] Cross-encoder loaded.
```

```
tfidf_lr = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1,2), min_df=2, max_features=
    ("clf",   LogisticRegression(max_iter=300, multi_class="multinomial"
])
tfidf_lr.fit(train["statement"], train["y"])
val_pred_lr = tfidf_lr.predict(valid["statement"])
print("[TFIDF-LR] Validation:\n", classification_report(valid["y"], val_

# Learning curve (macro-F1)
train_sizes, train_scores, cv_scores = learning_curve(
    tfidf_lr, train["statement"], train["y"], cv=3, scoring="f1_macro",
    train_sizes=np.linspace(0.1,1.0,5), n_jobs=-1
)
plt.figure(figsize=(6,4))
plt.plot(train_sizes, train_scores.mean(axis=1), marker="o", label="Trai
plt.plot(train_sizes, cv_scores.mean(axis=1), marker="o", label="CV F1 (
plt.xlabel("Train size"); plt.ylabel("F1 (macro)"); plt.title("Learning
plt.legend(); plt.tight_layout(); plt.show()
```

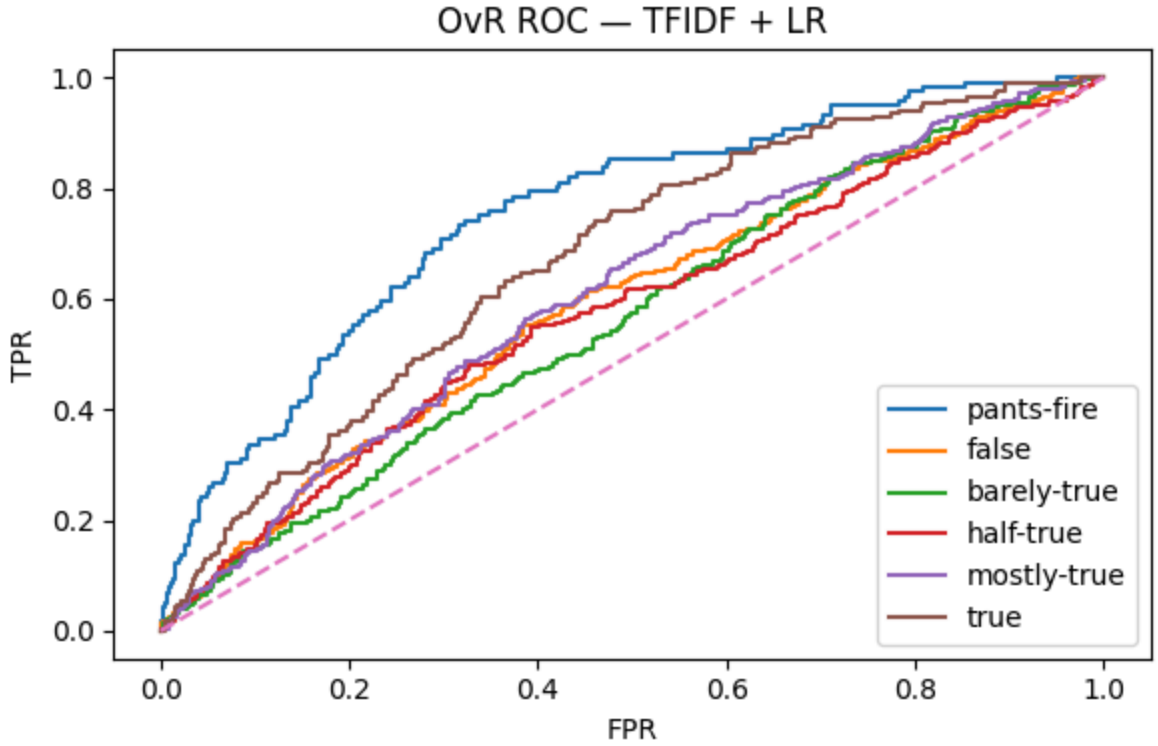
```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py
  warnings.warn(
[TFIDF-LR] Validation:
              precision    recall  f1-score   support

  pants-fire       0.40      0.07      0.12       116
       false       0.26      0.32      0.29       263
 barely-true       0.23      0.16      0.19       237
   half-true       0.25      0.35      0.29       248
 mostly-true       0.28      0.28      0.28       251
        true       0.21      0.21      0.21       169

    accuracy                           0.25      1284
   macro avg       0.27      0.23      0.23      1284
weighted avg       0.26      0.25      0.24      1284
```

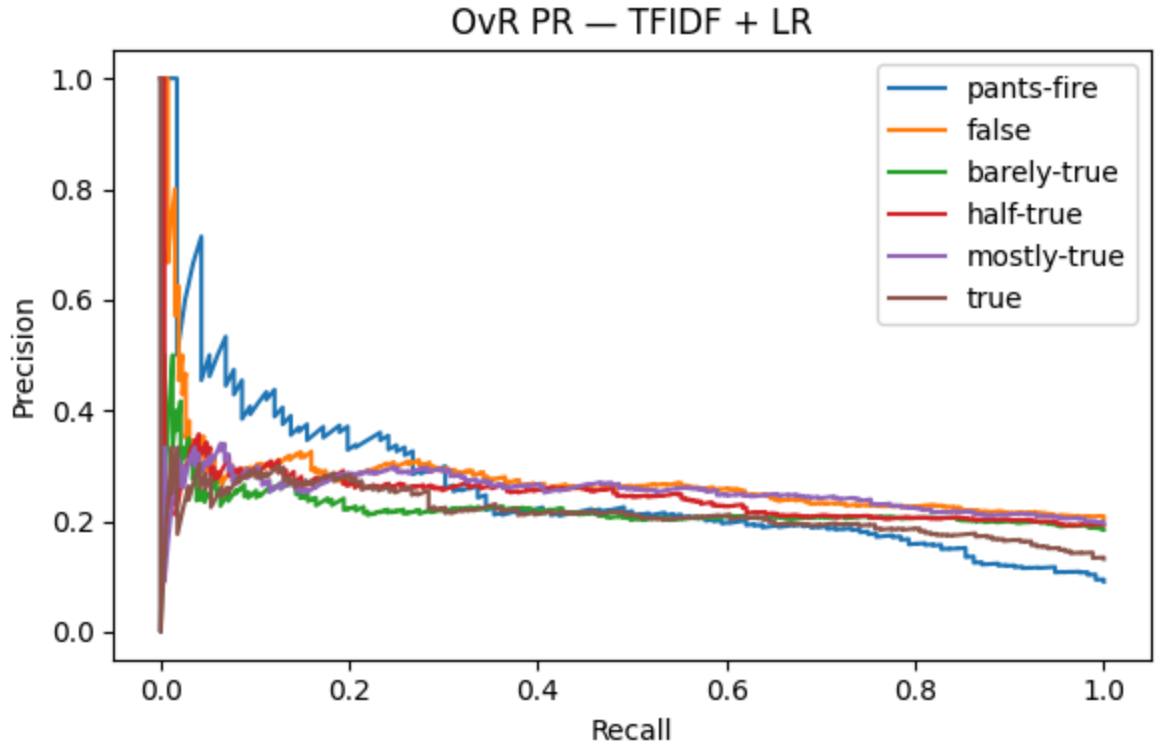



```
if hasattr(tfidf_lr, "predict_proba"):
    proba_lr = tfidf_lr.predict_proba(valid["statement"])
    from sklearn.preprocessing import label_binarize
    y_true = valid["y"].values
    y_bin = label_binarize(y_true, classes=list(range(len(LABELS))))
    try:
        roc_auc = roc_auc_score(y_bin, proba_lr, average="macro", multi_
        pr_auc  = average_precision_score(y_bin, proba_lr, average="macr
        print(f"[TFIDF-LR] Macro ROC-AUC={roc_auc:.3f}  Macro PR-AUC={pr
    except Exception as e:
        print("AUC computation issue:", e)
```

```
# OvR ROC per class
plt.figure(figsize=(6,4))
for i, lab in enumerate(LABELS):
    fpr, tpr, _ = roc_curve(y_bin[:, i], proba_lr[:, i])
    plt.plot(fpr, tpr, label=lab)
plt.plot([0,1],[0,1],"--")
plt.xlabel("FPR"); plt.ylabel("TPR"); plt.title("OvR ROC — TFIDF + L

# OvR PR per class
plt.figure(figsize=(6,4))
for i, lab in enumerate(LABELS):
    pr, rc, _ = precision_recall_curve(y_bin[:, i], proba_lr[:, i])
    plt.plot(rc, pr, label=lab)
plt.xlabel("Recall"); plt.ylabel("Precision"); plt.title("OvR PR — T
```

```
[TFIDF-LR] Macro ROC-AUC=0.628  Macro PR-AUC=0.246
```



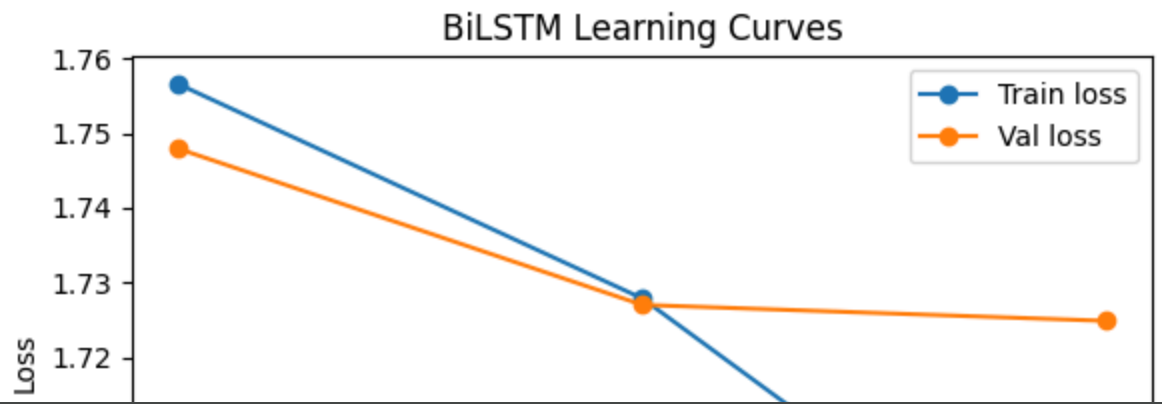



```
# === BiLSTM text classifier ===
MAX_VOCAB = 20000; MAX_LEN = 64; EMB_DIM = 100; HIDDEN = 128; EPOCHS = 3

def tok_words(s: str): return re.findall(r"[A-Za-z0-9_]+", str(s).lower(

# Build vocab
```

```
from collections import Counter
cnt = Counter()
for s in train["statement"].tolist(): cnt.update(tok_words(s))
vocab = {w:i+2 for i,(w,c) in enumerate(cnt.most_common(MAX_VOCAB))}
vocab["<pad>"]=0; vocab["<unk>"]=1

def encode(s: str, max_len=MAX_LEN):
    ids = [vocab.get(t,1) for t in tok_words(s)]
    ids = ids[:max_len] + [0]*(max_len - len(ids[:max_len]))
    return np.array(ids, dtype=np.int64)

class TextDS(Dataset):
    def __init__(self, df):
        self.x = np.stack([encode(s) for s in df["statement"].tolist()])
        self.y = df["y"].astype(int).values
    def __len__(self): return len(self.y)
    def __getitem__(self, i):
        return torch.from_numpy(self.x[i]), torch.tensor(self.y[i], dtyp

tr_ds, va_ds = TextDS(train), TextDS(valid)
tr_dl = DataLoader(tr_ds, batch_size=BATCH, shuffle=True)
va_dl = DataLoader(va_ds, batch_size=BATCH)

class BiLSTM(nn.Module):
    def __init__(self, vocab_size, emb_dim, hidden, n_labels):
        super().__init__()
        self.emb = nn.Embedding(vocab_size, emb_dim, padding_idx=0)
        self.lstm = nn.LSTM(emb_dim, hidden, bidirectional=True, batch_f
        self.drop = nn.Dropout(0.2)
        self.fc = nn.Linear(hidden*2, n_labels)
    def forward(self, x):
        e = self.emb(x); o,_ = self.lstm(e)
        m = self.drop(o.mean(dim=1))
        return self.fc(m)

bilstm = BiLSTM(len(vocab), EMB_DIM, HIDDEN, len(LABELS)).to(DEVICE)
opt = torch.optim.Adam(bilstm.parameters(), lr=LR)
crit = nn.CrossEntropyLoss()

def eval_dl(model, dl):
    model.eval(); ys=[]; ps=[]; pr=[]
    with torch.no_grad():
        for xb, yb in dl:
            xb = xb.to(DEVICE); yb = yb.to(DEVICE)
            logits = model(xb)
            prob = torch.softmax(logits, dim=-1).cpu().numpy()
            pr.append(prob); ps.append(logits.argmax(-1).cpu().numpy());
    y = np.concatenate(ys); p = np.concatenate(ps); proba = np.concatena
    return y, p, proba

tr_losses, va_losses, va_f1 = [], [], []
```

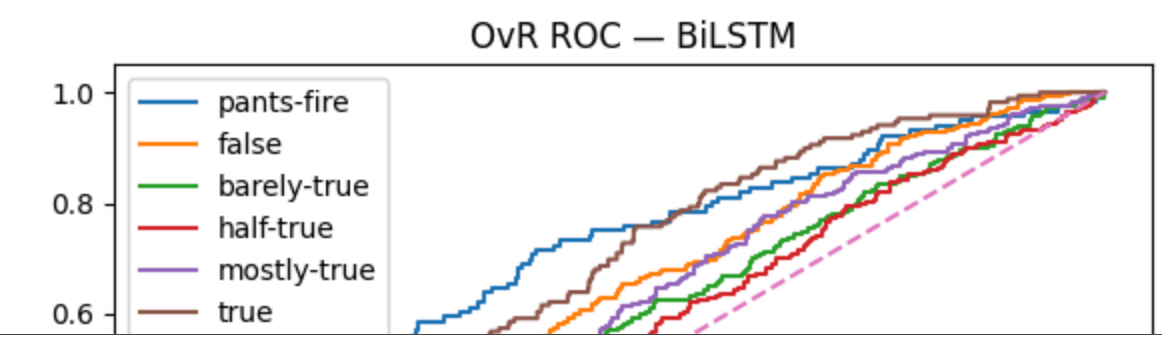
```
for ep in range(1, EPOCHS+1):
    bilstm.train(); tl=0; n=0
    for xb, yb in tr_dl:
        xb = xb.to(DEVICE); yb = yb.to(DEVICE)
        opt.zero_grad(); logits = bilstm(xb); loss = crit(logits, yb)
        loss.backward(); opt.step(); tl += loss.item()*yb.size(0); n +=
    yv, pv, proba = eval_dl(bilstm, va_dl)
    vl = 0.0
    with torch.no_grad():
        for xb, yb in va_dl:
            xb = xb.to(DEVICE); yb = yb.to(DEVICE)
            vl += crit(bilstm(xb), yb).item()*yb.size(0)
    f1m = f1_score(yv, pv, average="macro")
    tr_losses.append(tl/n); va_losses.append(vl/len(yv)); va_f1.append(f
    print(f"[BiLSTM] epoch {ep}: train_loss={tl/n:.4f} val_loss={vl/len(

plt.figure(figsize=(6,4))
plt.plot(range(1,EPOCHS+1), tr_losses, marker="o", label="Train loss")
plt.plot(range(1,EPOCHS+1), va_losses, marker="o", label="Val loss")
plt.xlabel("Epoch"); plt.ylabel("Loss"); plt.title("BiLSTM Learning Curv

print("[BiLSTM] Validation:\n", classification_report(yv, pv, target_nam

from sklearn.preprocessing import label_binarize
y_bin = label_binarize(yv, classes=list(range(len(LABELS))))
try:
    roc_auc = roc_auc_score(y_bin, proba, average="macro", multi_class="
    pr_auc  = average_precision_score(y_bin, proba, average="macro")
    print(f"[BiLSTM] Macro ROC-AUC={roc_auc:.3f}  Macro PR-AUC={pr_auc:.
except Exception as e:
    print("AUC computation issue:", e)

plt.figure(figsize=(6,4))
for i, lab in enumerate(LABELS):
    fpr, tpr, _ = roc_curve(y_bin[:, i], proba[:, i])
    plt.plot(fpr, tpr, label=lab)
plt.plot([0,1],[0,1],"--"); plt.xlabel("FPR"); plt.ylabel("TPR"); plt.ti
```

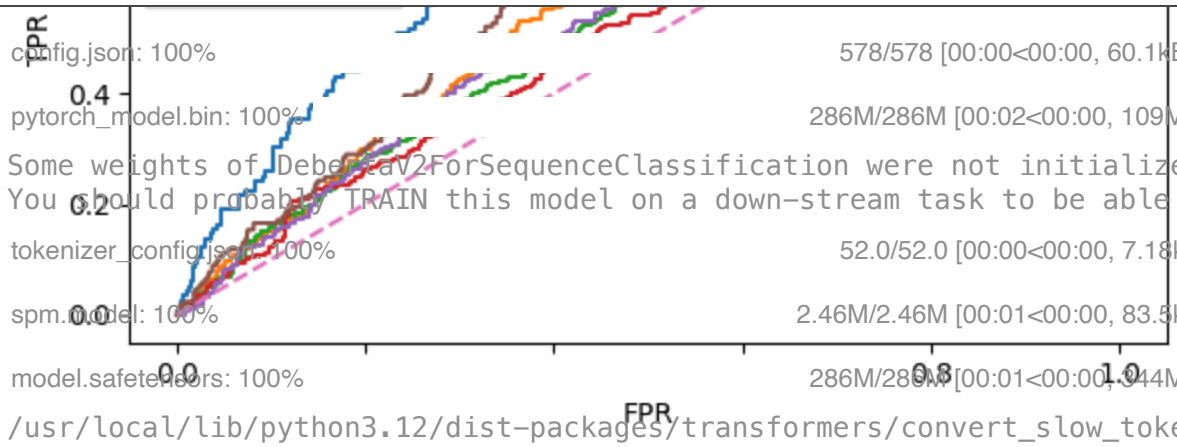
```
[BiLSTM] epoch 1: train_loss=1.7565 val_loss=1.7479 val_f1=0.106
[BiLSTM] epoch 2: train_loss=1.7279 val_loss=1.7270 val_f1=0.157
[BiLSTM] epoch 3: train_loss=1.6824 val_loss=1.7249 val_f1=0.188
```




```
# === OPTIONAL: Transformer fine-tune (DeBERTa) ===
MODEL_NAME = "microsoft/deberta-v3-small"
if TRANSFORMERS_AVAILABLE:
    try:
        model_t = AutoModelForSequenceClassification.from_pretrained(MOD
        tok = AutoTokenizer.from_pretrained(MODEL_NAME)
        def tok_fn(b): return tok(b["statement"], truncation=True, paddi
        tr_ds = HFDataset.from_pandas(train[["statement","y"]].rename(co
        va_ds = HFDataset.from_pandas(valid[["statement","y"]].rename(co
        args = TrainingArguments(
            output_dir=str(ROOT / "runs" / "liar_deberta"),
            num_train_epochs=1, per_device_train_batch_size=8, per_devic
            evaluation_strategy="epoch", save_strategy="epoch", load_bes
        )
        trainer = Trainer(model=model_t, args=args, train_dataset=tr_ds,
        print("[INFO] Transformer prepared. Uncomment to train:")
        print("trainer.train()")
    except Exception as e:
        print("[WARN] Transformer setup skipped:", e)
else:
    print("[INFO] Transformers not available — skipping.")
```

```
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result)
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result)
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result)
```






```
config.json: 100%                                   578/578 [00:00<00:00, 60.1kB/s]
pytorch_model.bin: 100%                             286M/286M [00:02<00:00, 109MB/s]
Some weights of DebertaV2ForSequenceClassification were not initialized f
You should probably TRAIN this model on a down-stream task to be able to
tokenizer_config.json: 100%                         52.0/52.0 [00:00<00:00, 7.18kB/s]
spm.model: 100%                                     2.46M/2.46M [00:01<00:00, 83.5kB/s]
model.safetensors: 100%                             286M/286M [00:01<00:00, 144MB/s]
/usr/local/lib/python3.12/dist-packages/transformers/convert_slow_tokeniz
  warnings.warn(
Map: 100%                                           10240/10240 [00:00<00:00, 13598.63 examples/s]
Asking to truncate to max_length but no maximum length is provided and th
Map: 100%                                           1284/1284 [00:00<00:00, 20150.06 examples/s]
[WARN] Transformer setup skipped: TrainingArguments.__init__() got an une
```

```
def sigmoid(x: float):
    try:
        return 1 / (1 + math.exp(-x))
    except OverflowError:
        return 0.0 if x < 0 else 1.0

def lookup_cred(row: pd.Series, table: pd.DataFrame, granularity: str =
    key = str(row.get(granularity, "unknown"))
    sub = table[table[granularity] == key]
    if len(sub) == 0:
        return 0.5
    return float(sub["credibility"].iloc[0])

def fuse_probs(clf_proba: np.ndarray, ce: float, cred: float, w_ev=0.3,
    fused = clf_proba.copy()
    boost = (w_ev * sigmoid(ce)) * (0.5 + 0.5 * cred)  # uses credibilit
    decay_idx = [0, 1]; lift_idx = [4, 5]
    shift = min(boost, fused[decay_idx].sum())
    if shift > 0:
        take = shift * fused[decay_idx] / (fused[decay_idx].sum() + 1e-9
        fused[decay_idx] -= take
        fused[lift_idx] += [0.4 * shift, 0.6 * shift]
    return fused / (fused.sum() + 1e-9)

# ---- pid-safe retrieval metrics ----
def retrieval_metrics(split: pd.DataFrame, k=5) -> Dict:
    recalls, ndcgs = [], []
    for _, row in split.iterrows():
        pid = row.get("pid", None)
        # Skip rows without a numeric id
        if pid is None or (isinstance(pid, float) and np.isnan(pid)):
```

```
            continue
        gold = f"verdict:{int(pid)}"

        cands = bm25_retrieve(row["statement"], k=k)
        cids = [c[0] for c in cands]

        # Recall@k
        recalls.append(1.0 if gold in cids else 0.0)

        # nDCG@k (binary relevance: 1 for gold, 0 otherwise)
        rel = np.array([[1.0 if cid == gold else 0.0 for cid in cids]])
        scores = np.array([[k - i for i in range(len(cids))]])
        try:
            from sklearn.metrics import ndcg_score
            ndcgs.append(ndcg_score(rel, scores))
        except Exception:
            ndcgs.append(0.0)

    return {
        f"Recall@{k}": float(np.mean(recalls)) if recalls else 0.0,
        f"nDCG@{k}": float(np.mean(ndcgs)) if ndcgs else 0.0
    }

print("Retrieval (valid):", retrieval_metrics(valid, k=5))
```

```
Retrieval (valid): {'Recall@5': 1.0, 'nDCG@5': 0.9990357161270583}
```

```
# OPTIONAL image-text similarity (CLIP). If offline, this block is skipp
CLIP_READY = False
try:
    from transformers import CLIPProcessor, CLIPModel
    clip_model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32
    clip_proc  = CLIPProcessor.from_pretrained("openai/clip-vit-base-pat
    CLIP_READY = True
    print("[OK] CLIP loaded.")
except Exception as e:
    print("[INFO] CLIP not available:", e)

from PIL import Image

def image_text_similarity(image_path: str, text: str) -> Optional[float]
    if not CLIP_READY: return None
    try: image = Image.open(image_path).convert("RGB")
    except Exception: return None
    import torch
    inputs = clip_proc(text=[text], images=image, return_tensors="pt", p
    with torch.no_grad():
        logits = clip_model(**inputs).logits_per_text.squeeze().item()
    return float(logits)
```

```
def predict(user_text: Optional[str]=None, image_path: Optional[str]=Non
    assert user_text or image_path, "Provide user_text and/or image_path
    if user_text is None: user_text = "Describe this image"
    cands = bm25_retrieve(user_text, k=5)
    ranked = rerank_cross_encoder(user_text, cands, topk=1)
    ce = ranked[0][3] if len(ranked)>0 else 0.0
    clf_proba = tfidf_lr.predict_proba([user_text])[0] if hasattr(tfidf_
    # No source row here; treat credibility neutral for user input
    cred = 0.5
    fused = fuse_probs(clf_proba, ce, cred, w_ev=w_ev, w_cred=w_cred)
    pred = id2label[int(np.argmax(fused))]
    sim = image_text_similarity(image_path, user_text) if image_path els
    return {
        "text": user_text, "image": image_path,
        "top_evidence_id": ranked[0][0] if len(ranked)>0 else None,
        "cross_encoder_score": ce, "image_text_similarity": sim,
        "fused_proba": {LABELS[i]: float(fused[i]) for i in range(len(LA
        "prediction": pred
    }

# Example prediction (text-only)
print(json.dumps(predict(user_text="The unemployment rate decreased last
```